AR Network Architect: An Interactive Augmented Reality Visualization of Network Topologies and Routing Algorithms

Divyam Kumar Choubey
Roll No: 23103070
Department of Computer Science and Engineering
Contact: +91-7903156432
Email: divyam@nitmanipur.ac.in

***Abstract*—This proposal outlines the design and implementation of "AR Network Architect," an interactive Augmented Reality (AR) educational tool. The project addresses the difficulties computer science students face when learning dynamic network topologies and packet routing algorithms. By utilizing WebXR and Computer Graphics techniques—including real-time transformations, raycasting for physical plane detection, and 3D animations—the application allows users to project, build, and visualize computer networks directly onto physical surfaces (e.g., a laboratory desk). This spatial approach transforms static 2D textbook diagrams into observable, dynamic 3D flows.**

I. Introduction

Augmented Reality (AR) provides a unique opportunity to overlay digital information onto the real world, making it an exceptional tool for visualizing invisible computational processes. In computer networking, concepts like data communication, packet routing, and network layer interactions are inherently spatial but are traditionally taught using flat, static diagrams. This project proposes an AR application that brings network nodes and data flows into the physical environment.

II. Problem Statement

Understanding network behavior (e.g., how a packet routes through a Mesh vs. Star topology, or how a node failure affects the network) is a core challenge in Computer Science education.

- **Limitation of Existing Methods:** 2D diagrams cannot adequately simulate real-time data flow, collision domains, or dynamic pathfinding.
- **AR Solution:** AR allows students to physically walk around a projected 3D network, interacting with nodes and observing packet traversal from multiple angles, significantly improving spatial understanding.

III. Motivation

The primary motivation is to make the invisible visible. By placing a virtual router on a physical desk and watching glowing 3D "data packets" travel to a virtual server, the cognitive load required to understand abstract routing algorithms (like Dijkstra's shortest path) is drastically reduced. *Educational Problem → Tangible AR Visualization → Improved Conceptual Understanding.*

IV. Objectives

1) To develop an AR educational tool that allows users to place 3D network nodes (routers, switches, PCs) on physical surfaces.
2) To implement plane detection and raycasting for accurate virtual-to-real-world object placement.
3) To demonstrate Computer Graphics techniques including hierarchical 3D modelling, lighting, and animated packet routing.
4) To integrate a modern React/Tailwind CSS user interface over the AR canvas for hardware selection and analytics.
5) To evaluate the application's usability and educational effectiveness with computer science students.

V. Target Users

- **Primary:** University engineering and computer science students studying Data Communication and Computer Networks.
- **Assumed Knowledge:** Basic understanding of what a computer network is, but no advanced knowledge of routing protocols is required.

VI. Proposed Application

The application acts as a virtual sandbox for building networks.

- **Main Features:** Users scan their physical desk using their smartphone camera. Once a flat plane is detected, they can place 3D models of computers and routers onto the desk.
- **User Interaction:** Users draw connections between nodes by tapping them. They can then select a "Source" and "Destination" and watch an animated data packet route through their physical space.
- **Feedback:** If a connection is severed (user deletes a link), the system visually recalculates the route, changing the color of the packet to indicate a delay or drop.

VII. Computer Graphics Techniques

- **3D Modelling:** Low-poly meshes will be used for network hardware to ensure high performance on mobile devices.
- **Transformations & Viewing:** Continuous application of translation and scaling matrices to keep the virtual network anchored to the real-world desk, even as the user moves their camera.
- **Lighting and Materials:**
  - *Directional Lighting:* Matched to the real-world camera feed (if supported) to cast realistic shadows onto the physical desk.
  - *Materials:* Glowing emissive materials for data packets to make them highly visible against physical backgrounds.
- **Animation:** Mathematical interpolation (Lerping) to animate packets traveling smoothly along the geometric lines connecting the 3D nodes.

VIII. AR Technology Implementation

This project relies strictly on Augmented Reality.

- **Environment Detection:** Utilizing ARCore/ARKit (via WebXR), the application analyzes the camera feed to detect horizontal feature points, estimating a physical plane.
- **Object Placement:** "Hit-testing" is used to cast a ray from the user's screen into the camera feed, finding the intersection point on the detected physical plane to spawn 3D objects.
- **Integration:** Virtual objects are superimposed over the real-world camera feed, allowing users to move physically around the virtual network.

IX. System Architecture

The system architecture isolates the modern UI layer (React) from the 3D rendering (Three.js) and the device's hardware sensors.

X. User Flow, Algorithm, and Mathematics

A. User Flow

Start $\rightarrow$ Scan Physical Desk (Plane Detection) $\rightarrow$ Tap to place Nodes $\rightarrow$ Link Nodes $\rightarrow$ Select Source/Dest $\rightarrow$ Algorithm Calculates Path $\rightarrow$ Observe 3D Animation $\rightarrow$ Review Analytics on 2D Overlay.

B. Mathematics: Raycasting and Distance

To accurately place objects on a physical desk, a ray R is cast from the screen coordinates through the projection matrix into world space. The intersection with the detected AR plane yields the x, y, z coordinates for placement. Once placed, the distance between network nodes is calculated using Euclidean geometry:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2} \quad (1)$$

This distance d is used as the weight for edges in the routing graph, determining the shortest path for the data packet animations.

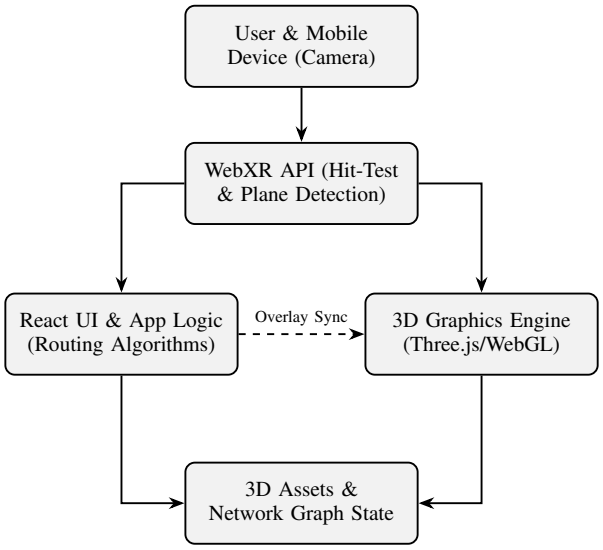

Fig. 1. System Architecture Diagram illustrating the interaction between the device camera, the AR hit-testing layer, the React frontend logic, and the 3D renderer.

XI. Proposed Implementation

The application will be developed as a Progressive Web App (PWA). The frontend UI will be built with React.js and Tailwind CSS, utilizing a glassmorphism aesthetic to ensure the camera feed remains visible behind UI elements. The 3D AR canvas will be managed by React-Three-Fiber (Three.js), hooking directly into the WebXR Device API for markerless AR tracking.

XII. Software/Hardware Requirements

- **Software:** React.js, Tailwind CSS, Three.js, WebXR Device API.
- **Hardware:** A smartphone or tablet equipped with AR capabilities (ARCore for Android, ARKit for iOS) and a compatible web browser (e.g., Chrome for Android).

XIII. Project Scope

In-Scope (Minimum Deliverables):

- Functional markerless AR plane detection and object placement.
- Ability to place at least three types of nodes (PC, Switch, Router).
- Visual animation of a packet travelling between nodes.
- A clean 2D user interface overlay for selecting tools.

Out-of-Scope: Complex packet inspection, real-world network traffic integration, and multi-user synchronized AR sessions are excluded to ensure timely completion.

XIV. Innovation

This project innovates by merging physical learning spaces with abstract computer science concepts. Rather than requiring students to mentally translate a flat textbook graph into a dynamic system, the AR environment physically maps the data structures onto their real-world surroundings, utilizing modern

web frameworks for seamless accessibility without app store installations.

XV. Testing and Evaluation Plan

- **Technical Evaluation:** Assessing AR tracking stability (how well objects stay anchored to the desk) and rendering frame rates (targeting 60 FPS) across different mobile devices.
- **Educational Effectiveness:** Conducting a user study with peers. Participants will be asked to identify network bottlenecks using a standard 2D diagram versus the AR visualization tool, measuring task completion time and accuracy.

XVI. Project Timeline

TABLE I
Proposed 7-Week Development Schedule

Phase	Activities
Week 1	UI/UX wireframing and project setup.
Week 2	WebXR initialization and plane detection.
Week 3	3D asset integration and Raycasting.
Week 4	Network graph logic and routing implementation.
Week 5	Packet animation and visual feedback.
Week 6	Testing on mobile devices, debugging tracking.
Week 7	Final documentation, demo video, and submission.

XVII. Expected Deliverables

1) This IEEE Format Project Proposal.
2) Source Code (React + Three.js) hosted on GitHub.
3) Functional AR Application Prototype.
4) Project Report and Testing Results.
5) Demonstration Video showing the AR application in use.

References

[1] M. Billinghurst, A. Clark, and G. Lee, "A survey of augmented reality," *Foundations and Trends in Human–Computer Interaction*, vol. 8, no. 2-3, pp. 73-272, 2015.

[2] W3C, "WebXR Device API," W3C Working Draft, 2023. [Online]. Available: https://www.w3.org/TR/webxr/

[3] Dirksen, J. *Learn Three.js - Fourth Edition*. Packt Publishing, 2023.

[4] Kurose, J. F., & Ross, K. W., *Computer Networking: A Top-Down Approach*. Pearson, 2021.